

Building a trading signal in Python

In stock markets, momentum is a persistence phenomenon that can be described in layman's terms as winners tend to continue to do well while losers continue to do badly. The conventional trading strategy is to buy assets that outperformed and sell assets that underperformed over a given period of time. Asset returns appear to exhibit negative autocorrelation over very short periods (less than one month) and longer time periods (more than three years) while the sweet spot for momentum, or positively auto-correlated, strategies is around 6-12 months.

We will use the ETFs in `Project Data.xls` to implement a momentum trading strategy. We will measure the performance of each asset using a moving average prices over short and long look-back periods.

Please see the Jupyter Notebook **Notebook_Sample_2.ipynb** that illustrates how some (but not all) of the steps below are implemented. Adapt this notebook to your project.

1. Upload the data with monthly returns of your ETFs in Python
2. Find the first date when all ETFs have data. Let's call this the start date. Assume that the initial price of each ETF on the start date is 100.
3. Calculate the monthly prices of your ETFs using the 100 starting value and the monthly returns. Make use of **cumprod()** property of the DataFrame.
4. Define the short-term and long-term look-back periods to be 2 and 4 months, respectively
5. Use the **rolling** property of the DataFrame that stores your ETF price data in step 3 to calculate the moving average price of each ETF over the short and long look-back periods, respectively.
6. Create a performance status variable that takes the values 0 or 1 for each ETF in your sample, based on how it performed in the short term vs long term. For a given ETF, the performance status variable takes the value 1 if the moving average price for the short-term is larger than the moving average price for long-term. For this step, use the **where** property in **numpy**.

7. Create a trading signal based on the change in performance status. For a given ETF and a given month, your trading signal is going to be the difference in the performance status. The trading signal can take three possible values:

- 1, corresponding to “buy”, when performance status changes from 0 to 1
- -1, corresponding to “sell”, when performance status changes from 1 to 0
- 0, corresponding to “hold”, when performance status doesn’t change (so either continues to be 0 or continues to be 1)

For this step use the **diff()** property of DataFrame that stores your performance status.

8. Create a trading strategy based on the trading signal in step 5. The trading strategy will indicate how the allocations in each ETF in your sample change from month to month. Assume you have \$1 million to invest in your strategy. Start with month 10 in your sample and initialize the weights of each of your ETFs that start with letter “V” at 10% and the ETF BSV at 40%. In month 11, we read the trading signal of each ETF in your sample and take action. If an ETF has a signal of hold, the position of that ETF remains unchanged (relative to the previous month). If an ETF has a signal of sell, we liquidate the position in that ETF and hold the proceeds in an asset called cash (the equivalent of the risk-free asset). Finally, if an ETF has a signal of buy, we use the cash to increase the position in that ETF. If only one ETF has a signal of buy, use all the cash available to increase the position of that ETF. If two or more ETFs have a buy signal, divide the cash equally across these ETFs to increase their positions. The next month (i.e., month 12), we repeat the process in month 11. We continue with this process until we reach the most recent month in our sample.
9. Plot the growth of your initial capital of \$1 million over the sample period. You can create these plots using **pyplot** in package **matplotlib**.